GRAND CHALLENGE - 2025

SoC Overview (FLOW OF RTL)

DEVELOPED BY :
SHAKTI DEVELOPMENT TEAM @ IITM
SHAKTI.ORG.IN

0.1 Proprietary Notice

The project was funded by the Ministry of Electronics and Information Technology (MeitY), Government of India

0.2 Release Information

Version	Date	Updates	Reviewed by
1.0	29/09/2025	Vignesh K, Satya	Kotteeswaran E
1.1	02/10/2025	Vignesh K, Satya	Kotteeswaran E
Draft V.1	07/10/2025	Vignesh K	Kotteeswaran E

TABLE OF CONTENTS:

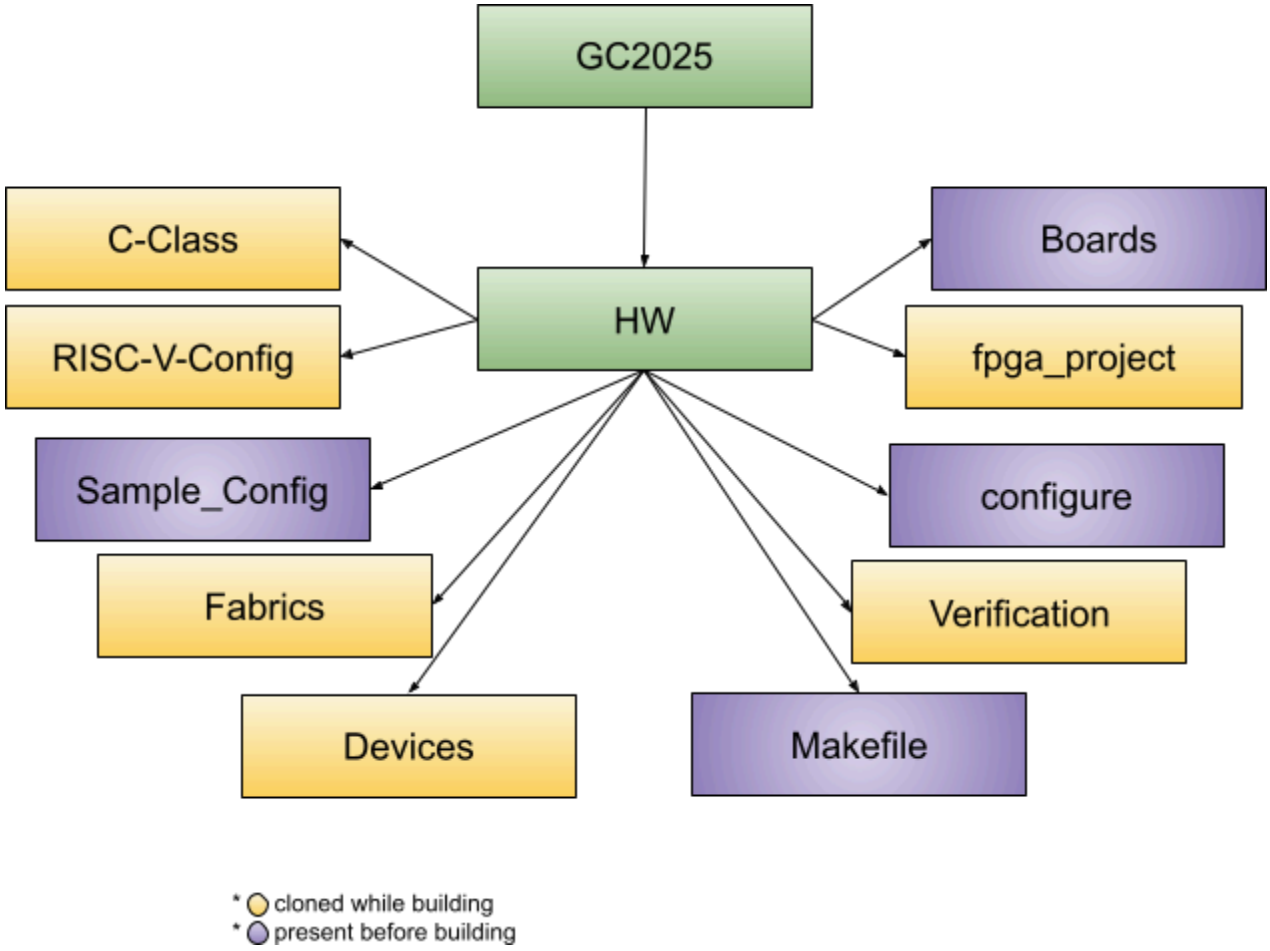
1. Introduction:

The rtl/bit stream generation for Shakti class of cores uses both bluespec system verilog and verilog files. Also it uses yaml files to configure the core and the peripherals. The bit stream generation uses the makefile flow of a typical linux terminal. This document details the organisation of all the above said params along with bit stream generation.

2. HARDWARE FLOW:

2.1. Overview:

In hardware, Core is built with the help of c-class, riscv-config, fabrics, boards, devices, sample_config and verification. On specifying the requirements according to the **Makefile,** the Core and Bitstream will be generated based on the requirements.In the RTL Flow, there are some major dependencies like Bluespec Compiler, Vivado and riscv-gnu-toolchain. Without these, RTL generation becomes impossible.



2.2. Makefile:

- Makefile plays an important role in maintaining the RTL flow process. Each and every make command helps in performing specific tasks like generating Verilog and IP's for the specific boards etc.
- Everything begins with a "make" command , the general make command like " **make build BOARD=arty_a7_yamuna**". In this every word other than make like "build" and BOARD=-- will build the core for a specific board, in this case ARTY A7.

2.3. Boards:

- Based on the board selection, the specific board and its files will be taken and the core will be built accordingly.
- So this board will have specific files such that it will generate a core according to the specifications(IP's and configurations etc).
- Each boards (hw/boards/[board_specific]) folder will consist of files for Soc, Debugging the SOC, UART, peripherals ,pwm and top.v as the top layer. This will also contain a file for memory mapping and .xdc for defining the ports and pins to be connected etc.
- They also contain '.tcl' files which are used to automate the process of IP generation, integration and Bitstream/MCS generation.

2.4. Configure:

- Shakti makes use of this Configure directory to configure the core with the help of python scripts and a schema file.
- This directory has scripts that contain all the extensions and repo's required for the core configuration and building.

2.5. Sample_config:

2.5.1. Core Configurations:

- Shakti configures the Core using .yaml files. There are c32 and c64 based files for 32-bit and 64-bit cores.
- Shakti uses specific .yaml files for core, control and status registers, customization, debugging and ISA .
- These .yaml files are configured into the SOC with the help of the python scripts from the Configure directory.

2.6. C-class:

- As this SOC is made with Shakti C-Class , this repo is cloned into the folder.
- The C-class directory contains all the architectural parts of the SOC in .bsv format like core, pipeline stages , branch predictor, mbox, fbox(if F extension is used) etc.
- This also contains a sample_config and some .yaml files for configuring a C-class core for a 32-bit or a 64-bit system.

2.7. RISC-V Config:

- This contains python scripts similar to what is there in the Configure directory in order to check and make the core RISC-V compliant, these scripts run and check them.

2.8. Fpga-project:

- This folder will be created by a .tcl script while generating bitstream.
- The Project folder contains the sources, runs and files of the Vivado project and also has a .xpr file to have a view of the project in a GUI format.

2.9. Verification:

- This directory contains a set of verification tests to check the core with the help of spike.
- There are tests from various categories like compliance,micro-arch level etc.

2.10. Devices:

- In devices, there are a wide set of peripherals which are developed by shakti.
- These are invoked when they are used in the core.
- UART, I2C, BRAM, DMA, GPIO, SPI, PWM etc. are some peripherals which are available in the directory.

2.11. Fabrics:

- Shakti is making use of AXI4 and AXI4lite interfaces which will be used to make connections in the SOC and connect peripherals with the SOC.
- These Interfaces will be present in the Fabrics directory.

2.12. Additional directories:

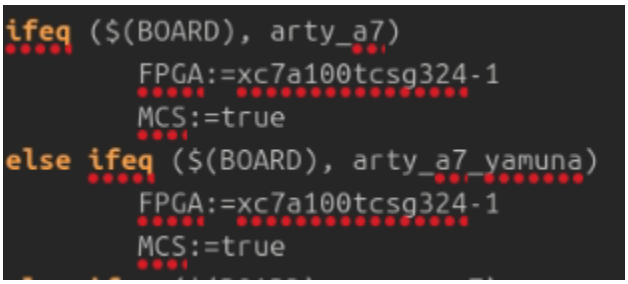
Directories	Use
CSRBOX	Configure the CSR registers in the core.
Caches_mmu	Configure the Caches in the core
common_verilog	Contains the pre-requisite verilog files for RTL flow.
common_bsv	Contains the pre-requisite bluespec files for RTL flow.

3. Adding New Boards and IP's:

Adding new boards and IP's can easily be done through some small changes which will be discussed below.

3.1. Changes for Adding New FPGA Boards:

- For adding new FPGA boards ,a folder needs to be created with a specific board name like arty_a7 ,consisting of all the necessary files like soc.bsv, fpga_top.v, tcl files etc according to the particular board configuration inside the boards folder in HW.
- Then changes in Makefile are required, like adding the board part with a set of if else statements.



- $BOARD will denote the board name specified here and this should match with the folder name to access files.
- In the same format ,add another board with a"**ifeq**" and FPGA containing the board part number like **xc7a100tcsg324**-1 and then based on the requirement whether MCS is required or not, if required then keep MCS as true else False.
- Please also check whether the Vivado has board files for the board part which is being added .
- adding core configuration for the board which is added in sample_config with either a 32 bit or 64 bit config for the particular board based on its requirements is advised.
- Another change would be there in Boot files where similar addition is done to generate boot files specific to the board

3.2. Changes for Adding New IP's through Verilog:

- For Adding new Verilog into the core building process, again go for the Makefile and under the division of generate_verilog and specify the path of the verilog file and the verilog file.

- These verilog files are copied to build ,while giving “make”.
- So after generation of bitstream,this specific IP will be added to the core .

3.3. Changes for Adding New Bluespec IP’s:

- For Adding new Bluspec IP’s, specify the path of the IP in bsvpath, a file which is present in HW.
- So on compiling these files in bsvpath, newer IP’s will be added to the core through bsvpath.
- For looking into integration of any other IP’s , please look into this

4. Building core and Generating Bitstream:

4.1. Installing Dependencies and copy board files:

- Before building, check whether the dependencies required are met and then board files are copied to the main folder for easy access.
- This is made possible by a requirements.txt file which is then read and checked through python.

4.2. Configuring the Core:

- Using Python scripts in Configure, core configurations for the specific boards are taken and configured according to the XLEN(bit size)(32 or 64) and boards.
- So in this case, there will be a c32_BOARD_name folder which will contain these .yaml files which will be configured to the core.

4.3. Build Directory:

- Build directory contains the IP’s and its verilog files. This build might change from design to design as there can be differences in verilog files due to different core and peripheral configurations.

4.4. Bluspec compilation:

- The Bluspec compiler will compile all the .bsv files involved in the process and will throw errors if there are any.
- Every code involved in that specific configuration will be compiled and will move on to generate verilog files.

4.5. Generate verilog and Boot Files:

- Commands like “make generate_verilog -j” and “make generate_boot_files” are used in this process.
- These commands will help in generating boot files required for the board and generate verilog files from the given bluespec library files.
- As boot files and other verilog files are required for the flow to progress, this step becomes a necessity.

4.6. Building IP's and Board:

- TCL files are used to optimize and automate the IP building and integration process.
- There are specific TCL files for automating IP builds for AXI, DDR, XADC etc.
- The commands like " make ip_build" & "make board_build" are used
- So these commands will invoke their TCL files like create_ip_project.tcl , create_project.tcl and run.tcl to create and import IP's which are required like DDR,XADC,Ethernet etc.
- Also to Map the IP's which are imported to the core configuration and check whether these match with each other and Map them according to the Board Specifications.
- Shakti has these in the form of .xci files like "mig_ddr3.xci" which will be imported and mapped with the core.
- With the help of run.tcl and env.tcl ,.xpr file is created for the project which can be used to view the project as a whole in Vivado.
- Also these tcl files help in measuring the timing and utilization constraints in the project.

4.7. Vivado Synthesis and Implementation:

- Using the create_project.tcl itself ,a Vivado project is created and initiates a Vivado synthesis and core implementation run which will help in making these verilog/blusepec code into RTL.
- Also the same RTL which is generated is converted into bitstream with the help of Vivado itself.

4.8. MCS generation:

- MCS is generated with the help of fpga_top.bit(bitstream) which is generated through this implementation and this is done with the help of generate_mcs.tcl which helps in converting this .bit into .mcs.
- .bit file could be found in the c-class.runs folder in the fpga-project directory.
- This MCS file or the Bitstream can be directly programmed to the board either through Vivado or program.tcl and further operations like connecting openOCD and starting a GDB server for debugging the core can be performed.